

Sistema de Gestão de Solicitações de Documentos Acadêmicos

Passo a passo para executar o projeto do zero — com e sem Docker — e navegar pelas telas.

Java 21 · Spring Boot 3.5 · PostgreSQL 17 · React 19 + Vite · JWT · JasperReports · Swagger

Este guia é a versão **ilustrada e resumida** do `README.md`. Os quadros pontilhados verdes são espaços reservados para os **prints do sistema**: salve cada imagem na pasta `docs/imagens/` com o nome indicado na legenda e gere o PDF novamente — a imagem entra sozinha no lugar do quadro.

1. Pré-requisitos

Cenário	Você precisa de	Conferir com
Com Docker (recomendado)	Docker Desktop 24+ (inclui Docker Compose) · Node.js 20+ e npm (para o frontend) · Git	<code>docker --version</code> <code>node --version</code>
Sem Docker	JDK 21 · PostgreSQL 17 · Node.js 20+ e npm · Git (o Maven não precisa ser instalado — o projeto traz o <code>mvnw</code>)	<code>java -version</code> <code>psql --version</code>

Dica: no modo com Docker você **não** precisa de Java nem de PostgreSQL instalados na máquina — o container cuida disso. O Node continua sendo necessário porque o frontend roda fora do container.

1 Clonar o projeto

```
git clone <url-do-repositorio> documentos-academicos
cd documentos-academicos
```

2 Configurar as variáveis de ambiente

Há dois arquivos `.env`: um para o backend (raiz) e um para o frontend.

```
# backend (na raiz do projeto)
cp .env.example .env

# frontend
cp frontend/.env.example frontend/.env
```

Importante: execute o comando `cp` — não copie/cole o texto do exemplo dentro do arquivo. Em seguida edite o `.env` da raiz e defina no mínimo:

- `DB_PASSWORD` — senha do banco;
- `JWT_SECRET` — chave com **32 caracteres ou mais** (sem ela a API nem sobe);
- `ADMIN_PASSWORD` — senha do usuário administrador criado no primeiro start (sem ela o sistema sobe sem nenhum usuário).

3 Subir a aplicação

Escolha **A** (com Docker) **ou B** (sem Docker). Os dois deixam a API em `http://localhost:8080`.

Opção A — com Docker (recomendado)

```
docker compose up --build -d

# confirme que subiu (deve responder {"status":"UP"}):
curl http://localhost:8080/actuator/health

# acompanhar logs / derrubar / apagar o banco:
docker compose logs -f
docker compose down
docker compose down -v
```

Opção B — sem Docker

Crie o banco no PostgreSQL uma única vez:

```
CREATE DATABASE documentos_academicos;
CREATE USER doc_user WITH PASSWORD 'a-mesma-do-DB_PASSWORD';
GRANT ALL PRIVILEGES ON DATABASE documentos_academicos TO doc_user;
```

O Spring Boot não lê o `.env` sozinho. Antes de subir, carregue as variáveis no terminal:

```
# Git Bash / Linux / macOS
set -a; source .env; set +a

# PowerShell
Get-Content .env | ForEach-Object {
```

```
if ($? -match '^s*([^#]=)(.*)$') { [Environment]::SetEnvironmentVariable($matches[1]
```

```
./mvnw spring-boot:run
```

Frontend (igual nos dois modos)

```
cd frontend  
npm install  
npm run dev           # abre em http://localhost:3000
```

4 Usando o sistema

Abra <http://localhost:3000> e entre com o `ADMIN_LOGIN` (padrão `administrador`) e o `ADMIN_PASSWORD` que você definiu.

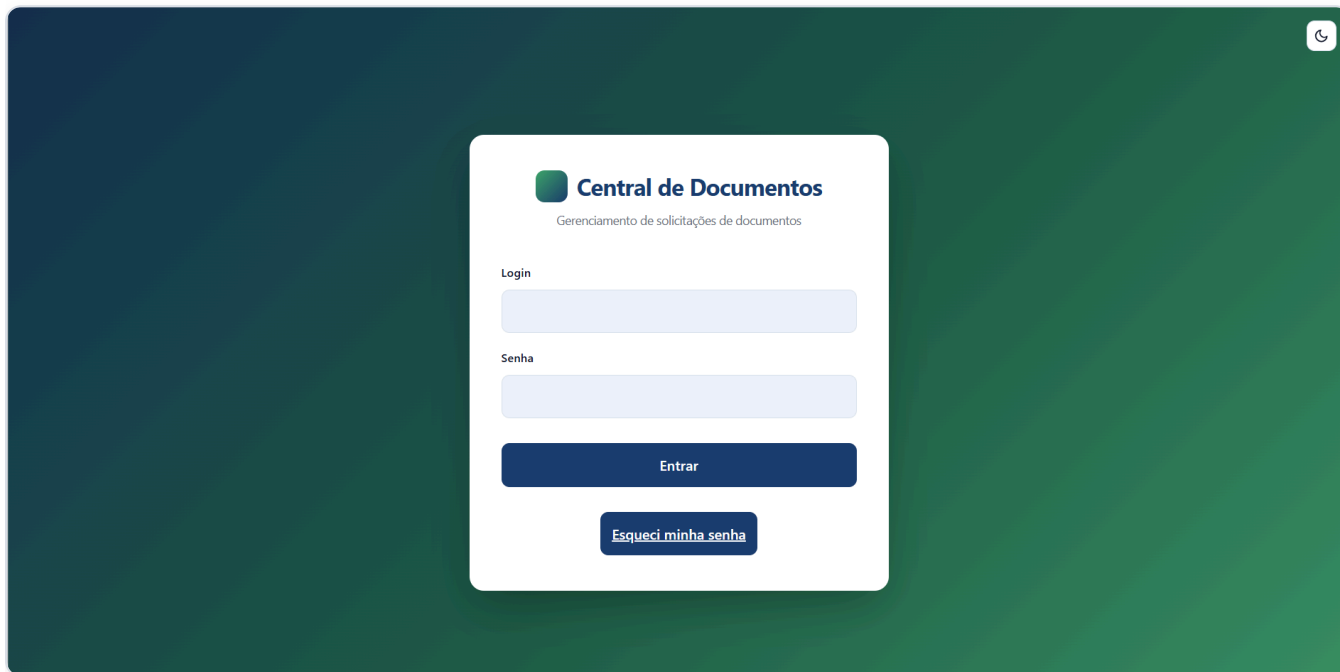


Figura 1 — Tela de login `docs/imagens/01-login.png`

Roteiro sugerido de primeiro acesso:

1. No menu **Cadastros**, crie ao menos um **aluno**, um **curso** e um **tipo de documento**.
2. Em **Solicitações** → **Nova**, monte a primeira solicitação usando os botões/chips de seleção.
3. Movimente o status (Em análise → Aprovada → Emitida) e acompanhe o fluxo.
4. Gere o **relatório PDF** pela listagem.

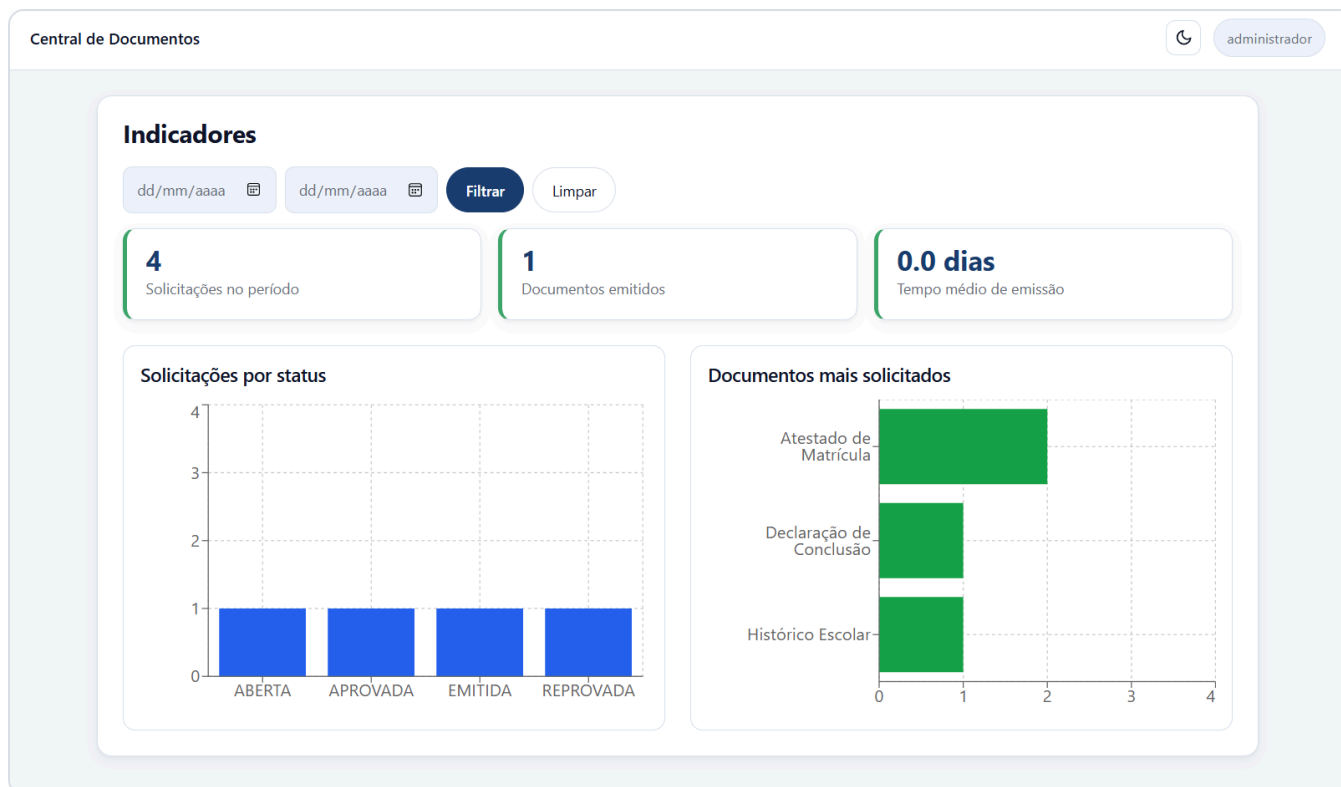


Figura 2 — Dashboard com os indicadores [docs/imagens/02-dashboard.png](#)

The "Nova solicitação" form is located within the "Central de Documentos" interface. It includes a "Voltar" button and several dropdown menus for selecting "Aluno", "Curso", and "Tipo de documento". Under the "Prioridade" section, there are three buttons: "Normal" (selected with a checkmark), "Alta", and "Urgente". A section for "Documentos anexos (opcional)" contains a "+ Adicionar arquivos" button. The form concludes with a "Criar solicitação" button.

Figura 3 — Formulário de nova solicitação (chips interativos) [docs/imagens/03-nova-solicitacao.png](#)

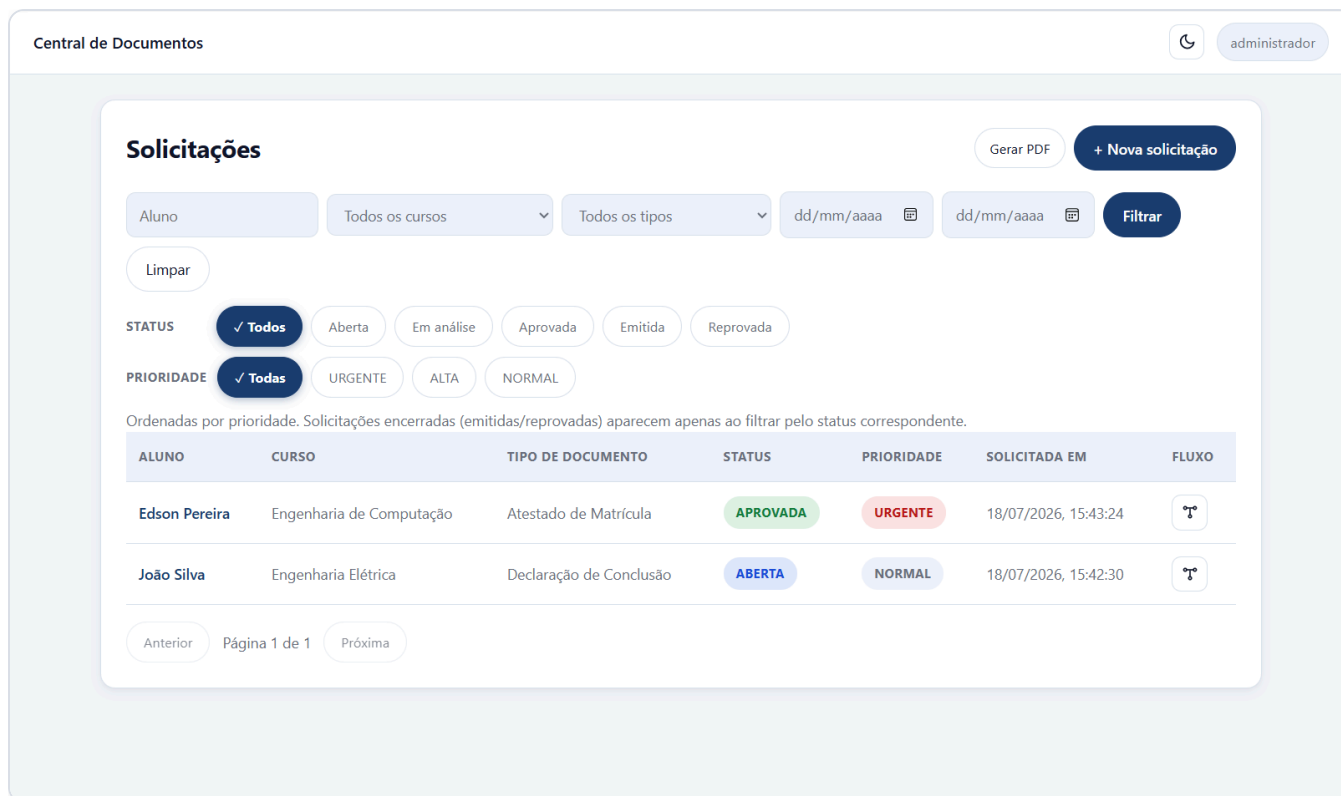


Figura 4 — Listagem de solicitações [docs/imagens/04-listagem.png](#)

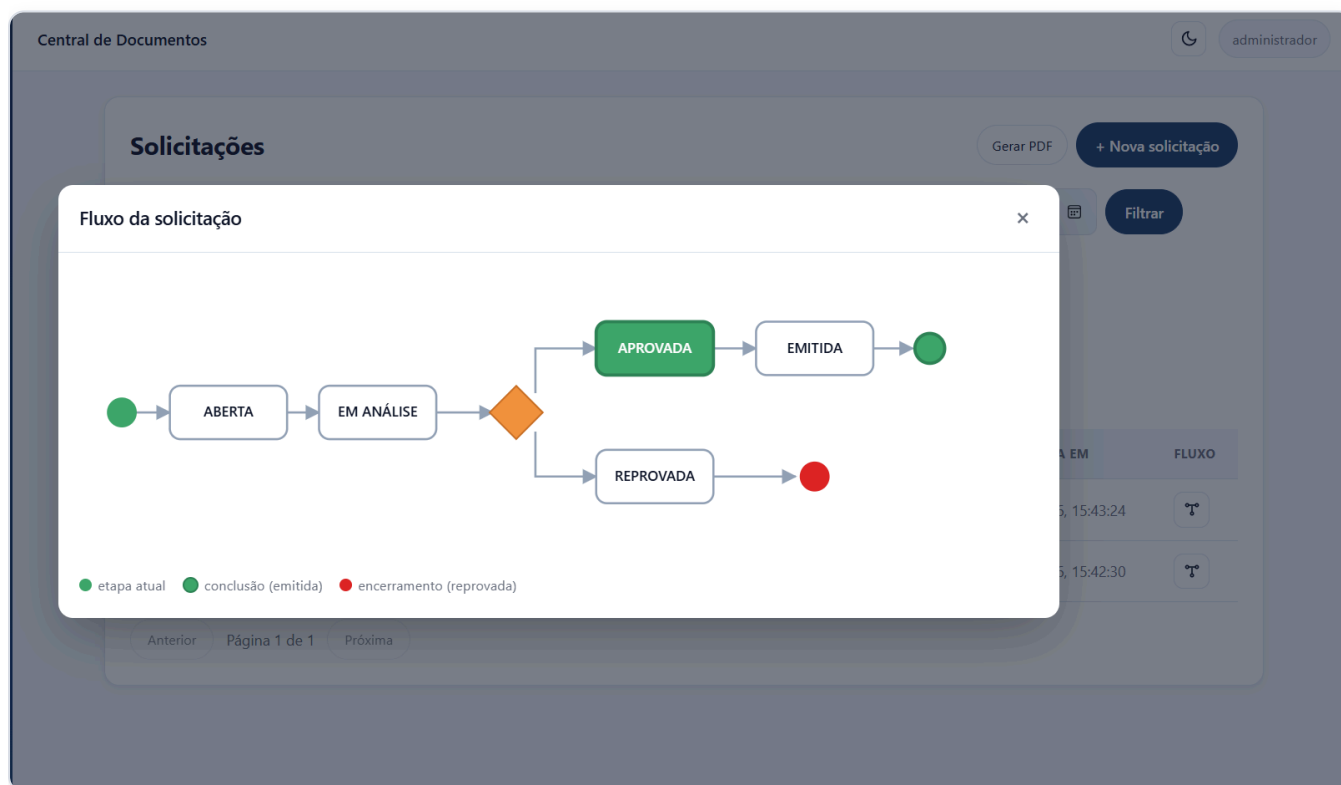


Figura 5 — Modal do fluxo (BPMN) destacando a etapa atual [docs/imagens/05-fluxo-bpmn.png](#)

Voltar

Solicitação #4

Ver fluxo

Aluno

Edson Pereira

Curso

Engenharia de Computação

Tipo de documento

Atestado de Matrícula

Status atual

Aprovada

Prioridade

URGENTE

Solicitada em

18/07/2026, 15:43:24

Última alteração

18/07/2026, 15:45:20

Mover para

Selecione o novo status...

Movimentar

Anexos

+ Anexar arquivo

teste.txt

27 B · 18/07/2026, 15:44:27

Baixar

Excluir

Histórico

DE	PARA	RESPONSÁVEL	QUANDO
—	Aberta	Samuel De Marco	18/07/2026, 15:43:24
Aberta	Em análise	Administrador	18/07/2026, 15:45:10
Em análise	Aprovada	Administrador	18/07/2026, 15:45:20

Figura 6 — Detalhe da solicitação com anexos docs/imagens/06-detalle.png

Relatório de Solicitações de Documentos

Gerado em: 18/07/2026 21:32

Filtros: nenhum (solicitações encerradas não incluídas)

ID	Aluno	Curso	Tipo de	Status	Solicitada
4	Edson Pereira	Engenharia de Computação	Atestado de Matrícula	Aprovada	URGENTE 18/07/26 15:43
3	João Silva	Engenharia Elétrica	Declaração de Conclusão	Aberta	NORMAL 18/07/26 15:42

Total de solicitações: 2

Figura 7 — Relatório PDF gerado docs/imagens/07-relatorio-pdf.png

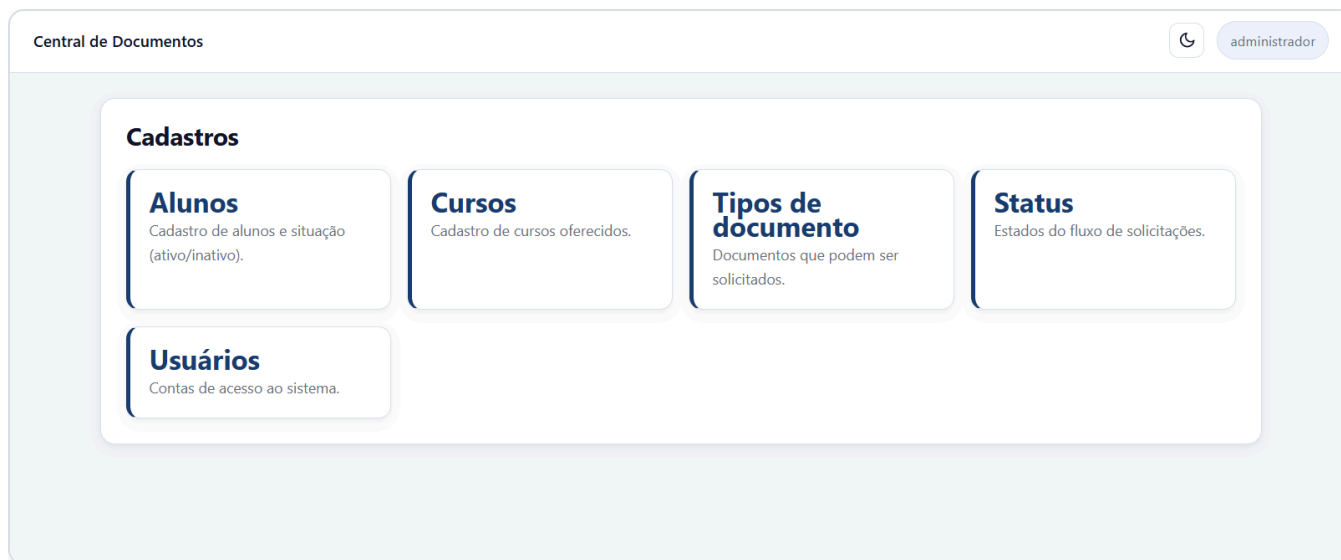


Figura 8 — Tela de cadastros básicos [docs/imagens/08-cadastros.png](#)

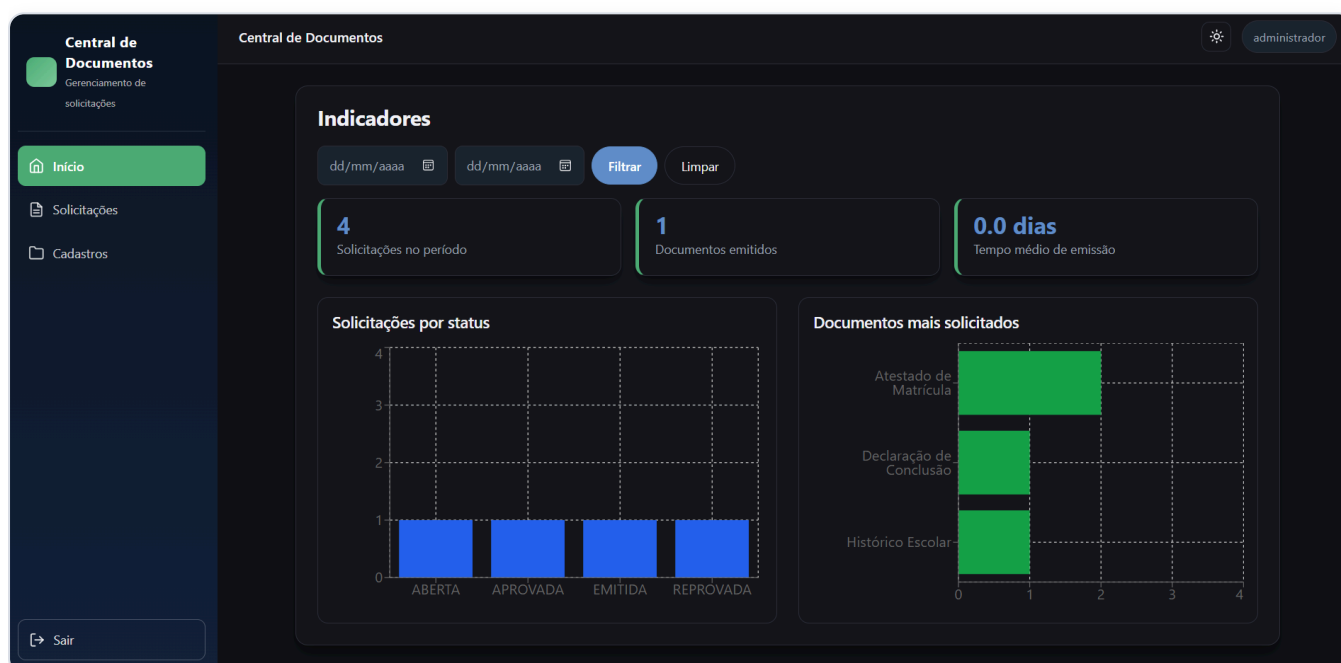


Figura 9 — Aplicação no modo escuro [docs/imagens/09-tema-escuro.png](#)

5 Documentação interativa (Swagger)

Com o backend rodando, o Swagger UI fica em:

Recurso	Endereço
Swagger UI	<code>http://localhost:8080/swagger-ui.html</code>
OpenAPI (JSON)	<code>http://localhost:8080/v3/api-docs</code>

Como testar um endpoint protegido:

1. Abra o Swagger e chame `POST /api/auth/login` com `login` e `senha` do admin — copie o `token` da resposta.
2. Clique em **Authorize** (cadeado, canto superior direito) e cole o token. Não precisa escrever "Bearer" — o esquema já é `bearer`.
3. Agora todos os endpoints usam o token; teste, por exemplo, `GET /api/solicitacoes`.

Observação: o Swagger fica **desligado no perfil prod** (é uma superfície de ataque desnecessária em produção). Ele está disponível nos perfis `dev` e `docker`.

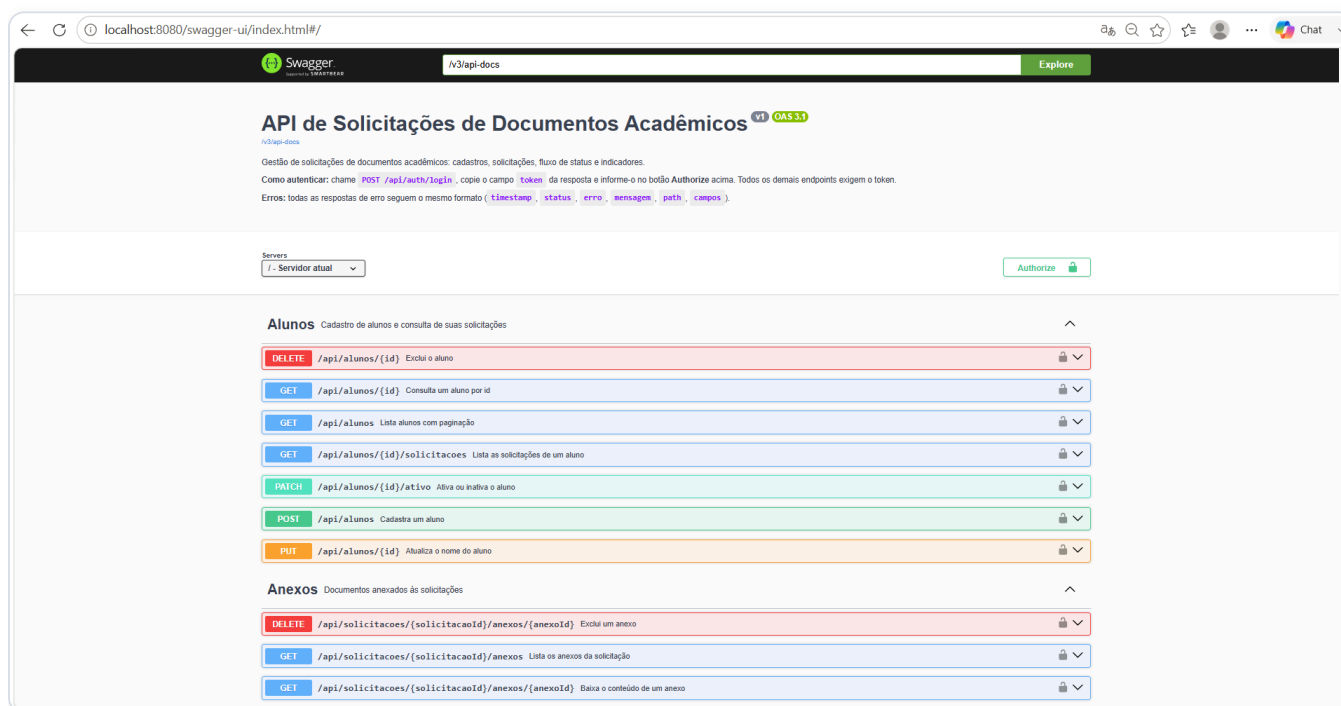


Figura 10 — Swagger UI docs/imagens/10-swagger.png

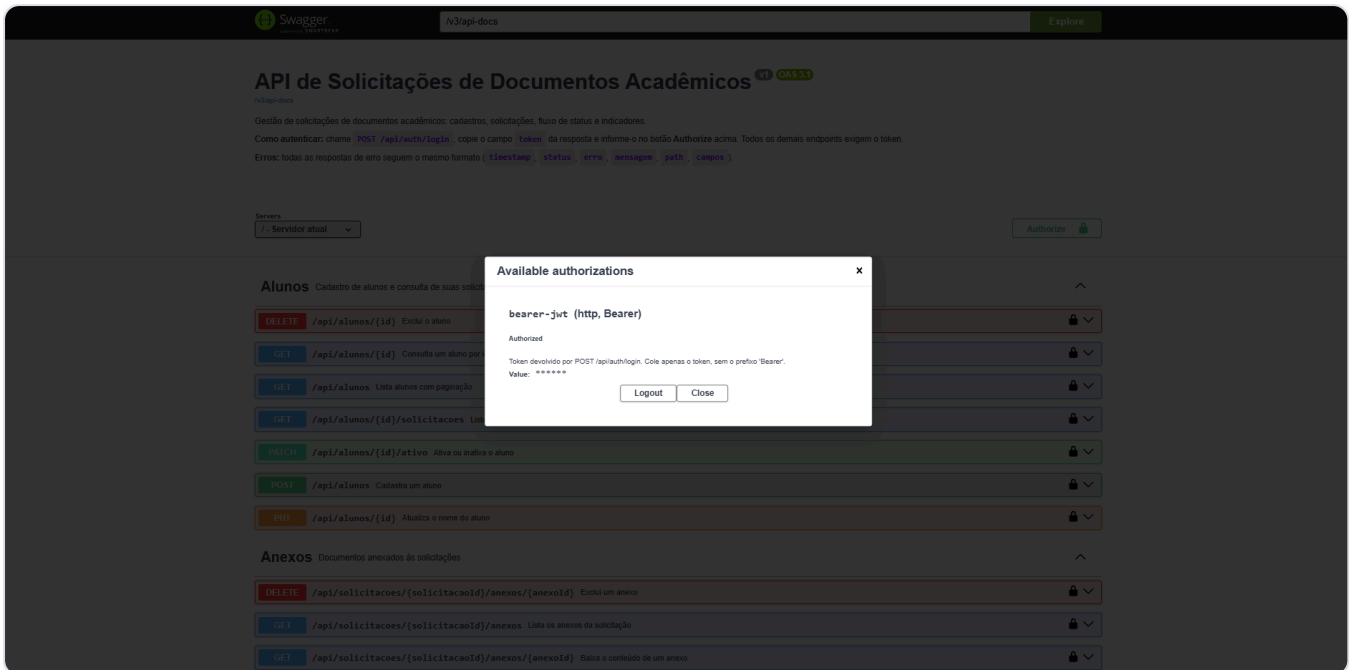


Figura 11 — Botão Authorize com o token JWT <docs/imagens/11-swagger-authorize.png>

6. Rodar os testes

```
# unitários (H2, rápido) - 104 testes
./mvnw test

# unitários + integração (Testcontainers, exige Docker) + análise estática
./mvnw verify

# frontend
cd frontend && npm run build && npm run lint
```

PARTE II

Como o sistema funciona

Perfis de acesso, o ciclo de vida de uma solicitação e o que cada tela faz.

7. Perfis de acesso

Todo acesso é autenticado por token JWT. Há três perfis, e o que cada um pode fazer é decidido **no servidor** (a interface só esconde botões por conveniência — a regra que vale é a da API).

Perfil	O que pode fazer
ADMIN	Tudo: cadastros (alunos, cursos, tipos, status, usuários), solicitações, movimentação de status, anexos e exclusões .
OPERADOR	Cria e movimenta solicitações, anexa e baixa documentos. Não acessa a aba de Cadastros nem exclui recursos.
CONSULTA	Somente leitura: visualiza solicitações, dashboard e relatórios. Não cria nem altera nada.

Regra geral no backend: escritas em cadastros exigem **ADMIN**; criar/mover solicitações e anexos exige **ADMIN ou OPERADOR**; exclusões (**DELETE**) são sempre **ADMIN**; leituras (**GET**) exigem apenas estar autenticado.

8. O ciclo de vida de uma solicitação

Cada solicitação de documento percorre uma sequência de **etapas (status)**. O caminho é fixo e validado pelo servidor — não é possível pular etapas nem voltar de uma etapa final.



1. ABERTA

O que é: estado inicial, criado automaticamente quando a solicitação é registrada.
Quem cria: ADMIN ou OPERADOR (informando aluno, curso, tipo de documento e prioridade).
Vai para: **EM ANÁLISE** (único caminho).

2. EM ANÁLISE

O que é: a solicitação está sendo avaliada por um responsável.
Vai para: **APROVADA** ou **REPROVADA** — é aqui que a decisão acontece.

3. APROVADA

O que é: pedido deferido, aguardando a emissão do documento.
Vai para: **EMITIDA**.

4. EMITIDA ✓ (final)

O que é: documento emitido — a solicitação está **concluída** e registra a data de emissão.
Vai para: nada. É um estado final e não se movimenta mais.

5. REPROVADA ✗ (final)

O que é: pedido indeferido na análise. Encerra a solicitação sem emissão.
Vai para: nada. Também é final.

As três regras que o servidor valida ao mover uma etapa

Toda mudança de status passa por três checagens, nesta ordem:

- Identidade:** quem move precisa informar o próprio `codigoResponsavel`; se não bater com o do usuário autenticado, a operação é recusada.
- Transição válida:** o destino precisa ser um passo permitido a partir da etapa atual (o diagrama acima). Pular ou voltar etapas é bloqueado (erro `422`).
- Responsável da etapa:** se a etapa de destino tiver um **responsável designado**, só esse responsável pode levar a solicitação até ela (erro `403`). Não há atalho de administrador aqui.

Como visualizar: na listagem e no detalhe há um ícone de **fluxo** que abre um modal em formato **BPMN**, destacando em qual etapa a solicitação está e para onde ela pode ir.

Prioridade e ordenação

Cada solicitação tem uma prioridade: **URGENTE**, **ALTA** ou **NORMAL**. A listagem já vem ordenada por prioridade (urgentes primeiro) e, dentro da mesma prioridade, das mais recentes para as mais antigas.

Solicitações encerradas

Para manter a tela focada no que precisa de ação, as solicitações **finalizadas** (EMITIDA e REPROVADA) **não aparecem** na listagem padrão — elas só surgem quando você filtra explicitamente por esse status.

9. O que cada tela faz

Login e recuperação de senha

O acesso é por **login e senha**. Na própria tela de login há o fluxo “**Esqueci a senha**”: o usuário informa o login, o sistema gera um **código de 6 dígitos** (válido por 15 minutos, uso único) e, com esse código, define uma nova senha.

Limitação assumida no projeto: não há serviço de e-mail, então o código de recuperação **volta na resposta** da API em vez de ser enviado por e-mail. É uma decisão documentada; em produção real o envio por e-mail seria o próximo passo.

Dashboard (indicadores)

Cartões de resumo, gráficos por **status** e por **período**, e os **documentos mais solicitados**, com um filtro de período que atualiza os indicadores. Serve para acompanhar o volume e a distribuição das solicitações.

Solicitações

É o coração do sistema. Nela você:

- **Lista e filtra** por aluno, curso, tipo de documento, status, prioridade e intervalo de datas (curso e tipo são listas do que está cadastrado).
- **Cria** uma nova solicitação escolhendo aluno, curso, tipo de documento e prioridade por botões interativos.
- Abre o **detalhe**, onde é possível **anexar documentos** (upload), baixá-los e — para ADMIN — removê-los.
- **Movimenta o status** seguindo as regras da seção 8, e consulta o **histórico** completo de movimentações (com autor e data, via auditoria).
- **Gera o relatório PDF** das solicitações, respeitando os mesmos filtros aplicados na tela.

Cadastros (somente ADMIN)

Telas para manter os dados de base: **alunos**, **cursos**, **tipos de documento**, **status** (onde se pode designar um **responsável** por etapa) e **usuários** (com seus perfis). As validações do servidor — como nomes duplicados — são exibidas ao usuário.

Tema claro / escuro

Um botão no topo alterna entre modo claro e escuro, e a preferência fica salva no navegador.